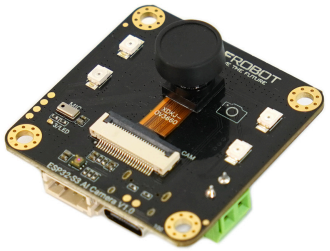
# ESP32-S3 AI CAM Wiki - DFRobot

## SKU:DFR1154 (https://www.dfrobot.com/product-2899.html)

(https://www.dfrobot.com/product-2899.html)



## README

- If you encounter situations such as failed code burning, inability to find the COM port, or the COM port continuously appearing and disappearing, please hold down the BOOT button, then click RST, and then download the code to successfully burn the code.

- All demo source codes and firmware can be downloaded at this link 【https://github.com/DFRobot/DFR1154_Examples】. You can quickly experience the functions and verify the hardware by burning the bin file. Tutorial on bin files (https://wiki.dfrobot.com.cn/FLASH_Download_Tool_Usage_Tutorial)
- The Gravity interface of V1.1 version has 3.3V output, so please do not connect to the input power supply.

# 1. Product Description

## 1.1 Product Introduction

The ESP32-S3 AI CAM is an intelligent camera module designed around the ESP32-S3 chip, tailored for video image processing and voice interaction. It is suitable for AI projects such as video surveillance, edge image recognition, and voice dialogue. The ESP32-S3 supports high-performance neural network computation and signal processing, equipping the device with powerful image recognition and voice interaction capabilities.

**Powerful AI Processing, Intelligent Image and Voice Recognition**
The ESP32-S3 AI CAM is powered by the ESP32-S3 chip, offering outstanding neural network computing capabilities. It can perform intelligent

computing capabilities. It can perform intelligent recognition on camera images and handle complex edge computing tasks. Additionally, the integration of a microphone and speaker enables support for

voice recognition and dialogue, facilitating remote command control, real-time interaction, and various AI functions. This makes it suitable for IoT devices and smart surveillance applications.

**Wide-Angle Infrared Camera for All-Weather Monitoring**

The wide-angle infrared camera on the ESP32-S3 AI CAM, combined with infrared illumination and a light sensor, ensures excellent image clarity even in low light or complete darkness. Whether day or night, the ESP32-S3 AI CAM guarantees the stability and clarity of surveillance footage, providing reliable support for security and monitoring systems.

**Voice Interaction and Control for Smart Automation**

With a built-in microphone and speaker, the ESP32-S3 AI CAM enables voice recognition and dialogue functionality. In smart home and IoT devices, this feature allows users to control the camera or other devices through voice commands, simplifying operations and enhancing the intelligent experience.

**Network Support for Expanding Online AI Functions**

In addition to local AI processing capabilities, the ESP32-S3 AI CAM can connect to the internet via

ESP32-S3 AI CAM can connect to the internet via Wi-Fi, extending more smart features through the cloud or online large models. Through Wi-Fi connectivity, this module can interface with cloud AI platforms to call upon large-scale language and vision models, enabling more advanced tasks such as sophisticated image classification, voice translation, and natural language dialogue. This not only empowers local computation but also taps into online resources for remote intelligent operations, significantly enhancing its potential in IoT devices.

## 1.2 Features

- Various AI capabilities
  - Edge image recognition (based on EdgeImpulse)
  - Online image recognition (openCV, YOLO)
  - Online large models for voice and image (ChatGPT)
- Equipped with a wide-angle night vision camera, infrared illumination, and all-day usability
- Onboard microphone and amplifier for voice interaction
- Offers a variety of AI models, with tutorial support for quick learning

## 1.3 Application Scenarios

- Surveillance cameras

- Surveillance cameras
- Electronic peephole
- AI assistant robots
- Time-lapse camera

# 2. Product Specifications

## 2.1 Product Parameters

### Basic Parameters

- Operating Voltage: 3.3V
- Type-C Input Voltage: 5V DC
- VIN Input Voltage: 3.7-15V DC
- Operating Temperature: -10~60°C
- Module Dimensions: 42*42mm

### Camera Specifications

- Sensor Model: OV3660
- Pixel: 2 Megapixels
- Sensitivity: Visible light, 940nm infrared
- Field of View: 160°
- Focal Length: 0.95
- Aperture: 2.0
- Distortion: <8%

### Hardware Information

- Processor: Xtensa® Dual-core 32-bit LX7 Microprocessor

- Main Frequency: 240 MHz
- SRAM: 512KB
- ROM: 384KB

- Flash: 16MB
- PSRAM: 8MB
- RTC SRAM: 16KB
- USB: USB 2.0 OTG Full-Speed Interface

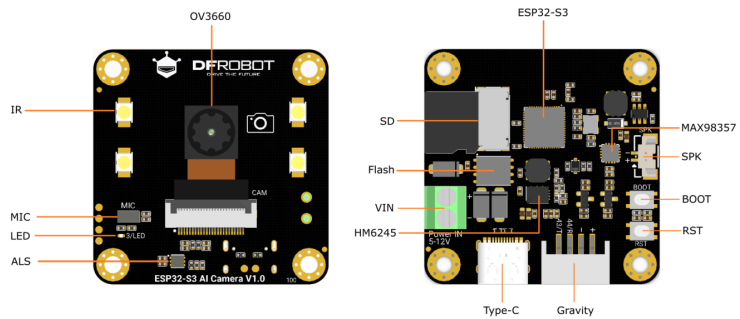
**Wi-Fi**

- Wi-Fi Protocol: IEEE 802.11b/g/n
- Wi-Fi Bandwidth: 2.4 GHz band supports 20 MHz and 40 MHz bandwidth
- Wi-Fi Modes: Station Mode, SoftAP Mode, SoftAP+Station Mode, and Promiscuous Mode
- Wi-Fi Frequency: 2.4GHz
- Frame Aggregation: TX/RX A-MPDU, TX/RX A-MSDU

**Bluetooth**

- Bluetooth Protocol: Bluetooth 5, Bluetooth Mesh
- Bluetooth Data Rates: 125 Kbps, 500 Kbps, 1 Mbps, 2 Mbps

## 2.2 On-board Function Diagram




- **OV3660**: 160° wide-angle infrared camera
- **IR**: Infrared illumination (GPIO47)
- **MIC**: I2S PDM microphone
- **LED**: Onboard LED (GPIO3)
- **ALS**: LTR-308 ambient light sensor
- **ESP32-S3**: ESP32-S3R8 chip
- **SD**: SD card slot
- **Flash**: 16MB Flash
- **VIN**: 3.7-15V DC input
- **HM6245**: Power chip
- **Type-C**: USB Type-C interface for power and code upload
- **Gravity**:
  - +: V1.0 has an input of 3.3-5V, and V1.1 has an output of 3.3V
  - -: GND

  - : GND
  - 44: GPIO44, native RX on ESP32-S3
  - 43: GPIO43, native TX on ESP32-S3
- **RST**: Reset button
- **BOOT**: BOOT button (GPIO0)
- **SPK**: MX1.25-2P speaker interface
- **MAX98357**: I2S amplifier chip

### 2.3 On-board Function Pin Definition

| CAM | GPIO | MIC | GPIO | MAX98357 | GPIO | ALS | GPIO | SD | GPIO | Other | GPIO |
|---|---|---|---|---|---|---|---|---|---|---|---|
| XCLK | 5 | PDM_CLK | 38 | BCLK | 45 | SDA | 8 | MO | 11 | LED | 3 |
| Y9 | 4 | PDM_DATA | 39 | LRCLK | 46 | SCL | 9 | MI | 13 | IR | 47 |
| Y8 | 6 | | | DIN | 42 | INT | 48 | SCLK | 12 | BOOT | 0 |
| Y7 | 7 | | | GAIN | 41 | | | CS | 10 | SDA | 8 |
| Y6 | 14 | | | MODE | 40 | | | | | SCL | 9 |
| Y5 | 17 | | | | | | | | | TX | 43 |
| Y4 | 21 | | | | | | | | | RX | 44 |
| Y3 | 18 | | | | | | | | | | |
| Y2 | 16 | | | | | | | | | | |
| VSYNC | 1 | | | | | | | | | | |
| HREF | 2 | | | | | | | | | | |
| PCLK | 15 | | | | | | | | | | |
| SIOD | 8 | | | | | | | | | | |
| SIOC | 9 | | | | | | | | | | |

## 3. Tutorial - First Time Use

### 3.1 Arduino IDE Configuration

When you use the ESP32 for the first time, you need to know the following steps:

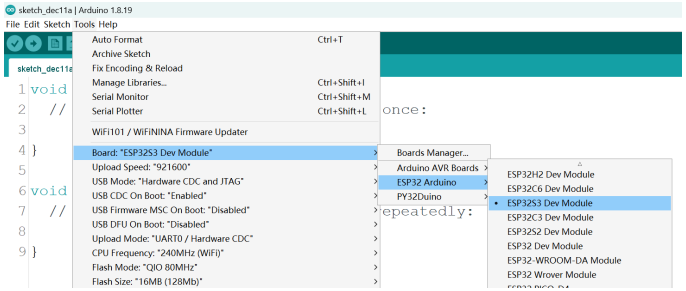
1. Add the ESP32 development board in Arduino IDE (How to add the ESP32 board to Arduino IDE?
(https://wiki.dfrobot.com.cn/Add_ESP32_board

(https://wiki.dfrobot.com.cn/Add_ESP32_board_to_Arduino_IDE?heisgoodman))

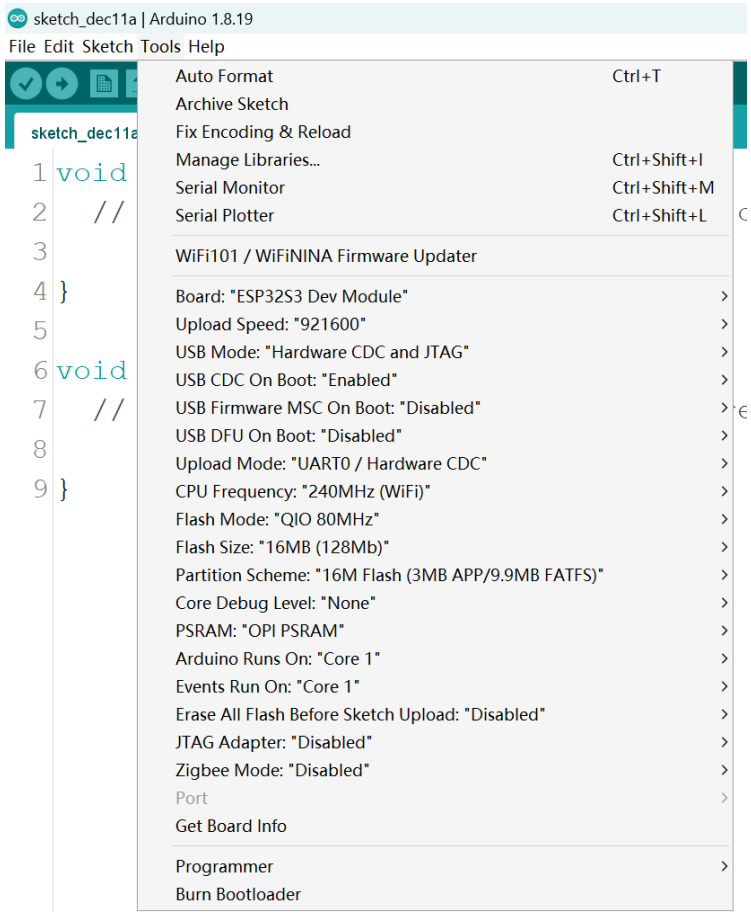
2. Select the development board and serial port
3. Burn the program

## 3.2 Select Development Board

- Click Tools->Board, select "ESP32S3 Dev Module".




- The development board needs to be set before burning the code:
  - USB CDC On Boot:
    - Enabled: Print serial port data through the USB interface
    - Disabled: Print serial port data through TX and RX
  - Partition Scheme: Disk partitioning scheme. Please select the appropriate storage space according to the Flash of the development board.
  - Port: Development board port (Just make sure the COM number is correct, which has nothing to do with the subsequent chip model.)




## 3.3 LED Blinking

- Copy the code into the window and click "**Upload**" to upload the code.

```
int led = 3;
void setup() {
  pinMode(led,OUTPUT);
}

void loop() {
  digitalWrite(led,HIGH);
  delay(1000);
  digitalWrite(led,LOW);
  delay(1000);
}
```

- Wait for the burning to complete, and you can see the on-board LED light start to flash.

    - If the LED light does not flash, reset the development board.
    - If the burning fails, please check Frequently Asked Questions ().

# 4. ESP32 General Tutorial

- FireBeetle 2 ESP32-S3 Basic Tutorial (https://wiki.dfrobot.com/SKU_DFR0975_FireBeetle_ESP32_S3_Basic_Tutorial)
- FireBeetle 2 ESP32-S3 Advanced Tutorial (https://wiki.dfrobot.com/SKU_DFR0975_FireBeetle_2_Board_ESP32_S3_Advanced_Tutorial)

# 5. Functional Examples

Functional examples can quickly verify whether the on-board functions of the development board are normal.

## 5.1 Acquiring Ambient Light Data

This example allows you to obtain ambient light data and print it via the serial port (baud rate:

data and print it via the serial port (baud rate: 115200).

**Note**: Please install the LTR308 Sensor Library (https://github.com/DFRobot/DFRobot_LTR308) before use.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

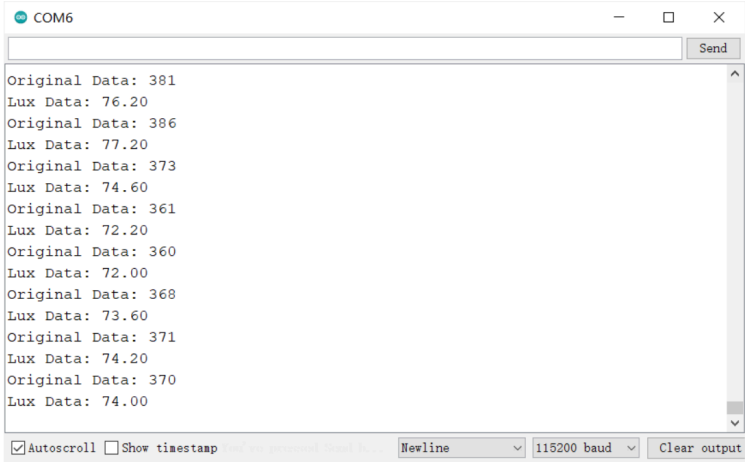
```
#include <DFRobot_LTR308.h>

DFRobot_LTR308 light;

void setup(){
    Serial.begin(115200);
    // If using a camera, place the came
    while(!light.begin()){
    Serial.println("Initialization faile
    delay(1000);
  }
  Serial.println("Initialization success
}
void loop(){
  uint32_t data = light.getData();
  Serial.print("Original Data: ");
  Serial.println(data);
  Serial.print("Lux Data: ");
  Serial.println(light.getLux(data));
```

```
  delay(500);
}
```

**Result**




## 5.2 Recording & Playback

This example enables recording and playback functions. After burning the code and resetting the development board:

1. The LED will light up to indicate recording for 5 seconds.
2. After the LED turns off, the recorded audio will play through the speaker.

**Note**: Ensure the speaker is properly connected.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

es)

```
#include <Arduino.h>
#include <SPI.h>

#include "ESP_I2S.h"

#define SAMPLE_RATE     (16000)
#define DATA_PIN        (GPIO_NUM_39)
#define CLOCK_PIN       (GPIO_NUM_38)
#define REC_TIME 5  //Recording time 5 s

void setup()
{
  uint8_t *wav_buffer;
  size_t wav_size;
  I2SClass i2s;
  I2SClass i2s1;
  Serial.begin(115200);
  pinMode(3, OUTPUT);
  i2s.setPinsPdmRx(CLOCK_PIN, DATA_PIN);
  if (!i2s.begin(I2S_MODE_PDM_RX, SAMPLE_
    Serial.println("Failed to initialize
  }
  i2s1.setPins(45, 46, 42);
  if (!i2s1.begin(I2S_MODE_STD, SAMPLE_R
    Serial.println("MAX98357 initializat
  }
```

```
  Serial.println("start REC");
  digitalWrite(3, HIGH);
  wav_buffer = i2s.recordWAV(REC_TIME, &
  digitalWrite(3, LOW);
  //Play the recording
  i2s1.playWAV(wav_buffer, wav_size);
}

void loop()
{

}
```

## 5.3 CameraWebServer

Burn the example code or bin file, then open the Serial Monitor (baud rate: 115200). Follow the prompts to enter your WiFi SSID and Password to connect to WiFi. Once connected:

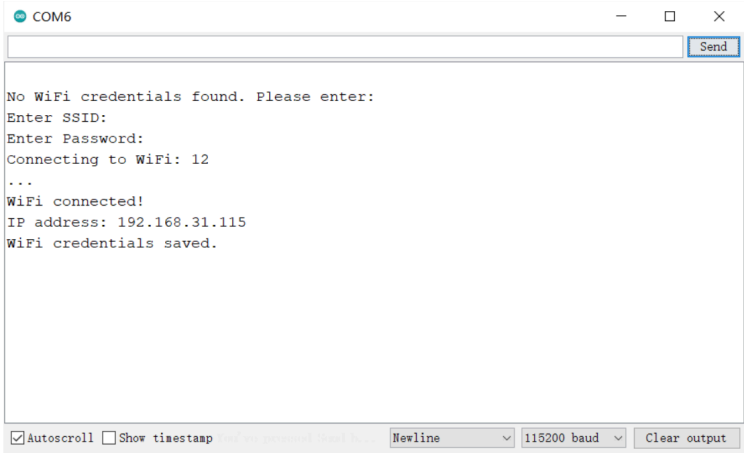
1. Access the IP address printed in the Serial Monitor via a web browser to reach the backend interface.
2. Click the "Start Stream" button to obtain the camera's live video feed.

**Note**: Ensure your device is on the same network as the development board.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Exampl

es)

**Result**




## 5.4 USBWebCamera

This code enables the ESP32-S3 to function as a virtual USB webcam, transmitting camera image data to a computer via USB. The computer can receive and display the images by running a Python script.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

**Steps**:

1. Burn the bin file to the module and connect it to your computer via USB.
2. Install Python (https://www.python.org/) if not

already installed.

3. Install the `opencv-python` library:
   - Press Win+R, type `cmd` to open the command prompt.
   - Run `pip install opencv-python` to install the library.
4. Open Python IDLE, go to `File > Open...`, select `Image_reception.py`, and press `F5` to run the script and view the camera feed.

**Notes**:

- If the window shows no image, your computer may have multiple cameras. Modify the `device_id` parameter in `Image_reception.py` and try again.
- The bin file is compiled from this example code, which requires ESP-IDF.
  Project URL:
  https://github.com/espressif/esp-iot-solution/tree/master/examples/usb/device/usb_webcam

# 6. Application Examples

## 6.1 Object Contour Recognition Based on Computer OpenCV

This example demonstrates how to use OpenCV on a computer to recognize object contours.

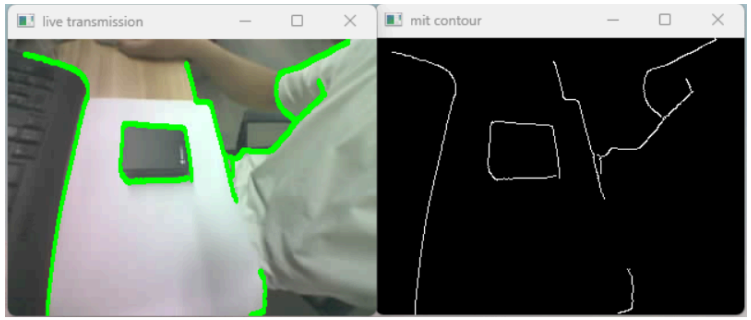
- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

### Steps

- Burn the code to the module (use the code from Section 5.3 or 5.4).
- Install the Python environment (https://www.python.org/). Skip this step if it is already installed.
- Install the opencv-python library:
  - Press Win+R, enter "cmd" to open the command window.
  - Enter "pip install opencv-python" to install the opencv-python library.
- Open Python IDLE, go to File->Open..., select the Python file (use "openCV_5_3.py" if you are using the example from Section 5.3; use "openCV_5_4.py" if you are using the example from Section 5.4). Press F5 in the new window that appears to see the image.

### Notes

- When using "openCV_5_3.py", remember to modify the IP address parameter.
- When using "openCV_5_4.py", remember to modify the camera_index parameter.




## 6.2 Object Classification Based on Computer YOLOv5

This example demonstrates how to use YOLOv5 on a computer for object classification.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

**Steps**:

1. Burn the code to the module (use the code from Section 5.3 or 5.4).
2. Install Python (https://www.python.org/) if not already installed.
3. Install required libraries:
   - Press Win+R, type `cmd` to open the command prompt.
   - Run `pip install opencv-python` to install

OpenCV.

- Run `pip install yolov5` to install the YOLOv5 library.

4. Open Python IDLE, go to `File > Open...`, select the appropriate Python file ( `yolo_5_3.py` for Section 5.3 example or `yolo_5_4.py` for Section 5.4 example), and press `F5` to run the script and view the results.

**Notes**:

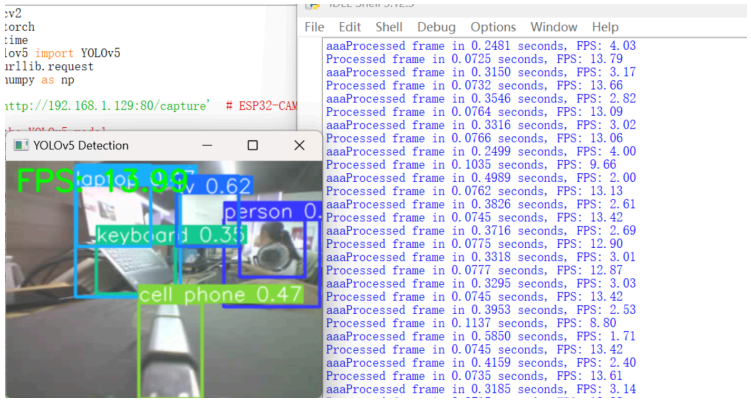
- When using `yolo_5_3.py`, modify the IP address parameter.
- When using `yolo_5_4.py`, modify the `cap = cv2.VideoCapture(0)` parameter to specify the correct camera index.

**What is the principle of object contour recognition?**

Object contour recognition involves detecting the boundaries of objects in an image by analyzing changes in pixel intensity. Common steps include:

1. **Preprocessing**: Convert the image to grayscale and apply noise reduction.
2. **Edge Detection**: Use algorithms like Canny to identify edges.
3. **Contour Extraction**: Connect edge points to form continuous contours.
4. **Postprocessing**: Filter and analyze contours

based on properties like area, perimeter, or shape.




## 6.3 Image Recognition Based on EdgeImpulse

This example uses the Person_Detection_inferencing model trained on EdgeImpulse for human detection. The recognition results can be viewed via the serial port, and the camera feed can be accessed through a web video stream.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

**Steps**:

1. Place the `Person_Detection_inferencing` folder into the `libraries` directory of your Arduino IDE installation.
2. Launch Arduino IDE, select `File > Examples >`

2. Launch Arduino IDE, select `File > Examples > Person_Detection_inferencing > edge_camera`, and burn the code to your device.

3. Open the Serial Monitor (baud rate: 115200), enter your WiFi SSID and Password as prompted to connect to WiFi.
4. After successful connection, access the IP address displayed in the Serial Monitor via a web browser to view the backend interface. Click "Start Stream" to obtain the live video feed.

## 6.4 Custom EdgeImpulse Models

For a tutorial on training EdgeImpulse models, visit the Wiki:
EdgeImpulse Object Detection Tutorial - DFRobot (https://wiki.dfrobot.com/EdgeImpulse_Object_Detection)

**Note**: Compilation errors may occur due to SDK version differences (demo uses SDK version 3.0.1).

## 6.5 Integration with HomeAssistant

Follow this tutorial to add the ESP32-S3 camera module to HomeAssistant and view the surveillance feed:
ESP32_S3_HomeAssistant (https://wiki.dfrobot.com/ESP32_S3_HomeAssistant)

## 6.6 Integration with Xiaozhi

This project is for accessing large AI models in the Chinese region. They may not be available in other regions.

## 6.7 Integration with OpenAI RTC

This example code allows real-time conversation with OpenAI (ChatGPT).

Project URL: https://github.com/DFRobot/openai-realtime-embedded-sdk

**Notes**:

- This code must run in the ESP-IDF environment.
- An OpenAI token is required for usage.

## 6.8 OpenAI Image Q&A

This example demonstrates the connection between DFRobot ESP32-S3 AI CAM and OpenAI for voice and image recognition. After burning the code:

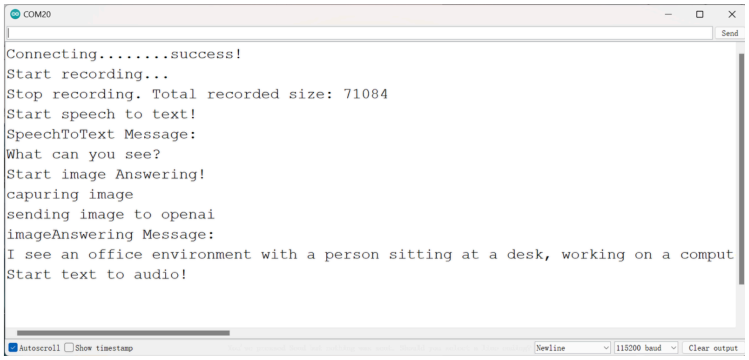
1. Hold the boot button and ask questions like "What do you see?", "What plant is this?", or "What is he doing?".
2. The module sends audio and image data to

2. The module sends audio and image data to OpenAI for recognition and plays the results via voice.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

**Notes**:

- An OpenAI token is required for usage.




## 6.9 Timed camera

This example demonstrates using the DFRobot ESP32-S3 AI Camera to take timed photos and save them to an SD card. The default photo interval is set to 25 seconds, which can be modified by adjusting the `TIME_TO_SLEEP` parameter. The LED will turn on when taking photos, and the ESP32-S3 will enter sleep mode after each photo is captured.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Exampl

es)

```
#include "esp_camera.h"
#include "FS.h"
#include "SD.h"
#include "driver/rtc_io.h"
#include "esp_sleep.h"

#define SD_CARD_CS   10

#define PWDN_GPIO_NUM     -1
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM     5
#define Y9_GPIO_NUM       4
#define Y8_GPIO_NUM       6
#define Y7_GPIO_NUM       7
#define Y6_GPIO_NUM       14
#define Y5_GPIO_NUM       17
#define Y4_GPIO_NUM       21
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM       16
#define VSYNC_GPIO_NUM    1
#define HREF_GPIO_NUM     2
#define PCLK_GPIO_NUM     15
#define SIOD_GPIO_NUM  8
#define SIOC_GPIO_NUM  9

#define TIME_TO_SLEEP  25 // Shooting in
```

```
#define uS_TO_S_FACTOR 1000000ULL
#define SLEEP_DURATION (TIME_TO_SLEEP *

RTC_DATA_ATTR int photo_count = 0; // Us

bool initCamera() {
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;
  config.pin_d0       = Y2_GPIO_NUM;
  config.pin_d1       = Y3_GPIO_NUM;
  config.pin_d2       = Y4_GPIO_NUM;
  config.pin_d3       = Y5_GPIO_NUM;
  config.pin_d4       = Y6_GPIO_NUM;
  config.pin_d5       = Y7_GPIO_NUM;
  config.pin_d6       = Y8_GPIO_NUM;
  config.pin_d7       = Y9_GPIO_NUM;
  config.pin_xclk     = XCLK_GPIO_NUM;
  config.pin_pclk     = PCLK_GPIO_NUM;
  config.pin_vsync    = VSYNC_GPIO_NUM;
  config.pin_href     = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn     = PWDN_GPIO_NUM;
  config.pin_reset    = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;
  config.frame_size = FRAMESIZE_UXGA;
  config.fb_location = CAMERA_FB_IN_PSRA
  config.jpeg_quality = 10;
  config.fb_count = 2;
```

```
  config.grab_mode = CAMERA_GRAB_LATEST;

  esp_err_t err = esp_camera_init(&confi


  sensor_t *s = esp_camera_sensor_get();
  // initial sensors are flipped vertica
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);        // flip i
    s->set_brightness(s, 1);   // up the
    s->set_saturation(s, -2);  // lower
  }

  return err == ESP_OK;
}

bool initSDCard() {
  if (!SD.begin(SD_CARD_CS)) {
    return false;
  }
  uint8_t cardType = SD.cardType();
  return cardType != CARD_NONE;
}

void takePhotoAndSave() {
  camera_fb_t * fb = esp_camera_fb_get()
  if (!fb) {
    Serial.println("Failed to obtain the
    return;
  }
```

```
  String path = "/photo_" + String(photo_
  fs::FS &fs = SD;
  File file = fs.open(path.c_str(), FILE_

  if (!file) {
    Serial.println("Save failed");
  } else {
    file.write(fb->buf, fb->len);
    Serial.println("Photo saving path: "
  }
  file.close();
  esp_camera_fb_return(fb);

  photo_count++; // The number of the ne
}

void setup() {
  Serial.begin(115200);
  delay(3000); // Give the serial port s

  if (!initCamera()) {
    Serial.println("The camera initiliz
    return;
  }

  if (!initSDCard()) {
    Serial.println("The initialization o
    return;
  }
  pinMode(3,OUTPUT);
  digitalWrite(3,HIGH);
```

```
  takePhotoAndSave();
  //delay(500);
  digitalWrite(3,LOW);

  Serial.println("Get ready to enter dee
  esp_sleep_enable_timer_wakeup(SLEEP_DU
  esp_deep_sleep_start();
}

void loop() {
  // It won't be executed up to here.
}
```

## 6.10 Play Online Music

This example demonstrates playing audio files from the network via ESP32-S3. After burning the code, enter the WiFi SSID and Password as prompted by the serial monitor to connect to WiFi. Then enter the network audio address to play the audio.

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)

**Note**

- Please install the ESP32-audioI2S library before using the example code. Library link: https://github.com/schreibfaul1/ESP32-audioI2S/

Copy

```
//*************************************
//*    audioI2S-- I2S audiodecoder for E
//*************************************
//
// first release on 11/2018
// Version 3  , Jul.02/2020
//
//
// THE SOFTWARE IS PROVIDED "AS IS" FOR
// FOR PERSONAL USE IT IS SUPPLIED WITHO
// WARRANTIES OF MERCHANTABILITY, FITNES
// OR COPYRIGHT HOLDER BE LIABLE FOR ANY
// OTHERWISE, ARISING FROM, OUT OF OR IN
//

#include "Arduino.h"
#include "WiFiMulti.h"
#include "Audio.h"
#include "SPI.h"
#include "SD.h"
#include "FS.h"

#include <WiFi.h>
#include <Preferences.h>

// Digital I/O used
```

```
#define SD_CS         10
#define SPI_MOSI      11
#define SPI_MISO      13
#define SPI_SCK       12
#define I2S_DOUT      42
#define I2S_BCLK      45
#define I2S_LRC       46

Audio audio;

Preferences preferences;
const char* ssid;
const char* password;

void connectToWiFi(const char* ssid, con
  WiFi.begin(ssid, password);
  Serial.printf("Connecting to WiFi: %s\
  int retries = 0;
  while (WiFi.status() != WL_CONNECTED &
    delay(500);
    Serial.print(".");
    retries++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi connected!");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\nFailed to connect
  }
```

```
}

void initWiFi() {

  preferences.begin("wifi", false);
  String savedSSID = preferences.getStri
  String savedPASS = preferences.getStri

  if (savedSSID.length() > 0 && savedPAS
    Serial.println("Found saved WiFi cre
    connectToWiFi(savedSSID.c_str(), sav

    if (WiFi.status() == WL_CONNECTED) {
      return;
    } else {
      Serial.println("Stored credentials
    }
  } else {
    Serial.println("No WiFi credentials
  }

  while (Serial.available()) Serial.read

  Serial.print("Enter SSID: ");
  while (Serial.available() == 0) delay(
  String inputSSID = Serial.readStringUn
  inputSSID.trim();

  Serial.print("Enter Password: ");
  while (Serial.available() == 0) delay(
  String inputPASS = Serial.readStringUn
```

```
  inputPASS.trim();

  connectToWiFi(inputSSID.c_str(), input


  if (WiFi.status() == WL_CONNECTED) {
    preferences.putString("ssid", inputS
    preferences.putString("password", in
    Serial.println("WiFi credentials sav
  } else {
    Serial.println("Failed to connect. C
  }

  preferences.end();
}

void setup() {
    pinMode(SD_CS, OUTPUT);
    digitalWrite(SD_CS, HIGH);
    SPI.begin(SPI_SCK, SPI_MISO, SPI_MOS
    SPI.setFrequency(1000000);
    Serial.begin(115200);
    SD.begin(SD_CS);

    initWiFi();

    audio.setPinout(I2S_BCLK, I2S_LRC, I
    audio.setVolume(15); // 0...21

//    audio.connecttoFS(SD, "test.wav");
//    audio.connecttohost("http://www.wd
```

```
//    audio.connecttohost("http://somafm
//    audio.connecttohost("http://mp3.ff
      audio.connecttohost("https://ra-sy

}

void loop(){
    vTaskDelay(1);
    audio.loop();
    if(Serial.available()){ // put strea
        audio.stopSong();
        String r=Serial.readString(); r.
        if(r.length()>5) audio.connectto
        log_i("free heap=%i", ESP.getFre
    }
}

// optional
void audio_info(const char *info){
    Serial.print("info        "); Serial
}
void audio_id3data(const char *info){  /
    Serial.print("id3data     ");Serial.
}
void audio_eof_mp3(const char *info){  /
    Serial.print("eof_mp3     ");Serial.
}
void audio_showstation(const char *info)
    Serial.print("station     ");Serial.
}
void audio_showstreamtitle(const char *i
```

```
    Serial.print("streamtitle ");Serial.
}
void audio_bitrate(const char *info){

    Serial.print("bitrate     ");Serial.
}
void audio_commercial(const char *info){
    Serial.print("commercial  ");Serial.
}
void audio_icyurl(const char *info){  //
    Serial.print("icyurl      ");Serial.
}
void audio_lasthost(const char *info){
    Serial.print("lasthost    ");Serial.
}
```

## 7. FLASH Download Tool Usage Tutorial

FLASH Download Tool Usage Tutorial (https://wiki.dfrobot.com/FLASH_Download_Tool_Usage_Tutorial)

## 8. MicroPython Tutorial

MicroPython Tutorial (https://wiki.dfrobot.com/ESP32_MicroPython_Usage_Tutorial)

# 9. PlatformIO Tutorial

PlatformIO Tutorial (https://wiki.dfrobot.com/ESP32_Platform_IO)

# 10. ESP-IDF Tutorial

Coming Soon

# 11. FAQ

- Q：What protocol does the MIC use?
  A：The MIC uses the PDM protocol. For more parameters of the MIC, please refer to the datasheet: https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/d2c58ceeebf0ab6a527b0c37c0ef525e.pdf

- Q：What is the specification of the speaker interface? What parameters of speaker are supported?
  A：The speaker interface uses an MX1.25mm interface, and the recommended speaker parameters are 4R1.5W and 8R1W.

- Q：Can the EdgeImpulse object classification model be customized?
  A：Yes, it is supported. Please refer to the training method at: https://wiki.dfrobot.com/EdgeImpulse_Object_Detection

- Q：What are the parameters of the infrared fill light?
  A：The maximum power of a single LED is

  75mW, the minimum radiation intensity is 5mW/sr, and the standard radiation intensity is 7mW/sr.

- Q：What is the maximum supported size of TF card?
  A：The maximum supported size is 32GB FAT32 formatted TF card.

## 12. Related Documents

- Demo source code and firmware (https://github.com/DFRobot/DFR1154_Examples)
- Schematic v1.0 (https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/97463680cd773f5a1493d24adfc1a4af.pdf)
- Schematic v1.1 (https://dfimg.dfrobot.com/62d52567aa9508d63a4247a1/wikien/62f7850198e513d61ce2e02ecde3eb42.pdf)
- Dimension Drawing (https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/7ad8fd0712eca1c4262661adf1d29723.pdf)

- OV3660 Specifications (https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/e56429a294e02939081f361ae6bacebc.pdf)
- OV3660 Datasheet (https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/2c067b2856a6ecd6a84d05cd289e449a.pdf)
- MIC Datasheet (https://dfimg.dfrobot.com/5d57611a3416442fa39bffca/wiki/d2c58ceeebf0ab6a527b0c37c0ef525e.pdf)

**Back to Top**